# rustai

Engine huấn luyện AI viết bằng Rust: từ tensor CPU đến GPU và học tăng cường

Ngày 11/11/2026 · Rust 1.99.1 · Windows 11 · Intel Core i5-13420H (12 luồng) · NVIDIA GeForce RTX 4050 Laptop · so sánh với PyTorch 2.6.0

## 1. Tóm tắt

Dự án xây một thư viện huấn luyện AI hoàn chỉnh bằng Rust theo sáu giai đoạn của lộ trình nghiên cứu, và kiểm chứng từng giai đoạn bằng các nguồn tham chiếu độc lập (PyTorch, NumPy, thư viện `safetensors` của Python, và chính executor CPU cho phần GPU). Cả sáu giai đoạn đã xong và vượt qua kiểm tra: 183 test Rust, 7 test Python, và 132 trong 134 lần phá code (mutation) bị bắt (hai lần còn lại là biến thể tương đương hành vi).

**Kết quả chính.** Một bước training được dựng thành đồ thị, tối ưu (CSE, fusion, loại code chết), lập kế hoạch bộ nhớ một lần rồi chạy lặp lại mà không cấp phát lần nào. Trên CPU, bước training MLP cỡ medium mất 1,06 ms, nhanh hơn PyTorch eager 1,9× (1 luồng) và 2,0× (8 luồng); ở cỡ nhỏ nhanh hơn 13,1×–25,5×. Kernel matmul CPU đạt 82–91% PyTorch (MKL) trên một luồng ở hầu hết shape. Backend GPU (WGSL qua wgpu) chạy toàn bộ tập op, đạt 1,9 TFLOP/s (39% cuBLAS) và huấn luyện mạng nhỏ nhanh hơn PyTorch CUDA 1,5×. PPO trên CartPole chạy 81.028 bước/giây, gấp 6,7× cài đặt PyTorch tương đương, với hiệu quả mẫu tương đương.

**Hạn chế quan trọng.** Lợi thế lớn nhất là chi phí điều phối thấp và kernel ghi sẵn vào buffer, trên bài toán MLP kích thước cố định, so với PyTorch eager. Ở mạng lớn trên GPU engine chậm hơn PyTorch CUDA 2,5–3,2× vì GEMM của nó chỉ bằng khoảng 35–45% cuBLAS. Chỉ có float32, chưa có convolution hay attention. Mọi số đo chạy trên một máy, lấy kết quả tốt nhất qua hai vòng xen kẽ, nên nên đọc các tỉ lệ như bậc độ lớn.

## 2. Bối cảnh và lộ trình

PyTorch chủ yếu dùng Python để điều khiển; phần tính toán nặng chạy bằng C++ và CUDA, nên chuyển vòng lặp training sang Rust không tự động nhanh hơn. Nghiên cứu ban đầu chỉ ra một hướng khả thi: engine biên dịch đồ thị, tối ưu cho các workload có chi phí điều phối lớn so với tính toán tensor (mô hình nhỏ đến trung bình, học tăng cường). Sáu giai đoạn của lộ trình đã hoàn tất:

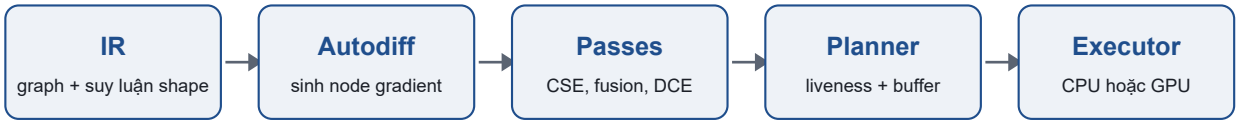
| Giai đoạn | Nội dung | Điều kiện hoàn thành | Trạng thái |
|---|---|---|---|
| 1 | Tensor CPU, broadcasting, GEMM | Phép toán đúng, benchmark nền | Xong |
| 2 | Autograd, optimizer, MLP | Gradient khớp PyTorch, MLP hội tụ | Xong |
| 3 | Graph IR, autodiff, CSE, fusion, memory planner, executor | Giảm allocation và thời gian mà không đổi kết quả | Xong |
| 4 | Backend GPU (WGSL qua wgpu), AdamW trên GPU | Chạy mọi op, khớp CPU, huấn luyện được trên GPU | Xong |
| 5 | Môi trường, GAE, PPO, action mask, shape bucketing | PPO học được; khớp PyTorch/NumPy; so với baseline | Xong |
| 6 | Checkpoint safetensors, Trainer, binding Python, ví dụ, CI | Tương thích hai chiều với Python; thư viện dùng được | Xong |

## 3. Kiến trúc

Mã nguồn là một workspace Cargo gồm các crate thư viện, một crate facade `rustai` làm điểm vào, một crate binding Python, một crate đối chiếu PyTorch và một crate benchmark. Thư mục đặt tên ngắn; tên package vẫn có tiền tố `rt-`. Số dòng tính theo dòng Rust khác rỗng, gồm cả test.

| Thư mục | Vai trò | Dòng |
|---|---|---|
| crates/tensor | Tensor f32, broadcasting, kernel ghi vào buffer cho sẵn, GEMM chia khối (AVX2/FMA) | 1.679 |
| crates/autograd | Engine động (tape), cài đặt độc lập thứ hai | 434 |
| crates/ir | Graph có kiểu, suy luận shape, autodiff sinh node IR | 679 |
| crates/compiler | Gỡ broadcast thừa, CSE, fusion (Linear, tích có mặt nạ), loại code chết | 627 |
| crates/memory | Phân tích vòng đời và lập kế hoạch tái dùng buffer | 208 |
| crates/runtime | Executor tham chiếu và executor planned không cấp phát | 570 |
| crates/gpu | Compute shader WGSL qua wgpu, AdamW trên thiết bị (thêm 291 dòng WGSL) | 1.389 |
| crates/nn, crates/optim | MLP, suy luận theo bucket, SGD, AdamW, cắt gradient | 697 + 251 |
| crates/rl | Môi trường, môi trường song song, GAE, PPO | 1.217 |
| crates/checkpoint | Đọc và ghi định dạng safetensors | 249 |
| crates/rustai | Facade: Trainer, re-export, prelude, ví dụ | 501 |
| crates/python | Binding PyO3 (buffer protocol, trả về NumPy) | 172 |
| tests/parity, benchmarks | Fixture PyTorch/NumPy và test đối chiếu; benchmark Rust và PyTorch | 729 + 373 |

Luồng biên dịch của một bước training:




Có thêm khoảng 1.100 dòng Python (fixture, test, benchmark, công cụ). Thư mục `tools/` có các script chạy kiểm tra từng giai đoạn, xuất ảnh kết quả test và biểu đồ benchmark.

# 4. Thiết kế chính

## 4.1 Kernel ghi vào buffer cho sẵn

Mọi phép toán nặng là hàm nhận slice vào và slice ra (`zip_into`, `matmul_into`, `reduce_into`, ...). Các phương thức của `Tensor` chỉ cấp phát đầu ra rồi gọi cùng kernel, nên mỗi phép toán chỉ có một cài đặt, dùng chung cho engine động, executor tham chiếu và executor planned. Vòng duyệt broadcasting dùng mảng stride cố định (tối đa 8 chiều) nên không cấp phát heap.

## 4.2 IR, autodiff và các pass

Mỗi node có op, đầu vào và shape; shape được suy luận và kiểm tra ngay khi thêm node. `gradients` duyệt ngược từ loss và thêm node gradient vào chính đồ thị. Các pass chạy theo thứ tự: gỡ `BroadcastTo` thừa; loại biểu thức con chung (có tính giao hoán); gộp `MatMul + bias + ReLU` thành một node `Linear` chạy trong epilogue của GEMM; gộp `MatMulNT + ReluGrad` thành một node dùng mặt nạ; và loại code chết. Mọi pass bảo toàn kết quả từng bit, vì epilogue chạy trên tích vô hướng đã hoàn tất, đúng thứ tự như các phép riêng lẻ. Việc thay mặt nạ `x > 0` bằng `y > 0` (với y = max(x, 0)) là chính xác.

## 4.3 Lập kế hoạch buffer và executor

Phân tích vòng đời cho mỗi node biết bước đọc cuối cùng; planner duyệt tuyến tính, chọn buffer rảnh nhỏ nhất đủ chỗ, nếu không thì mở rộng buffer rảnh lớn nhất. Executor cấp phát toàn bộ buffer một lần và chạy từng node bằng cách lấy tạm buffer đích ra khỏi kho (`std::mem::take`, không cần mã unsafe).

### 4.4 GEMM chia khối

Cấu trúc BLIS rút gọn: cả A và B được đóng gói thành tile liên tục trong buffer thread-local tái dùng; K chia khối 256; micro-kernel 6×16 (đường FMA) hoặc 4×8 (đường portable) giữ toàn bộ accumulator trong thanh ghi; chỉ chạy song song khi có hơn $2^{20}$ phép nhân-cộng. Mỗi phần tử đầu ra luôn cộng dồn theo thứ tự k tăng dần, nên kết quả không phụ thuộc chia khối, số luồng hay layout toán hạng; $A \cdot B^T$ và $A^T \cdot B$ (dùng ở backward) vì vậy giống hệt từng bit so với chuyển vị rồi nhân. Đường FMA được chọn lúc chạy bằng phát hiện CPU, qua đúng một khối `unsafe`.

### 4.5 Backend GPU

Mỗi op của IR ứng với một trong sáu compute shader WGSL (elementwise có broadcasting, unary, ternary, matmul, reduce, theo hàng) cộng một shader AdamW. GEMM dùng tile 64×64, mỗi luồng tính 4×4 phần tử, operand có stride tuỳ ý (nên NT/TN không tốn thêm) và epilogue bias/ReLU/mặt nạ. Executor GPU dùng lại pass tối ưu và memory planner của CPU. Tham số có thể nằm cố định trên thiết bị và được AdamW cập nhật tại chỗ, nên một bước training chỉ cần chạy các dispatch mà không chuyển dữ liệu qua CPU.

### 4.6 Học tăng cường

PPO loss là một đồ thị IR bình thường, dùng thêm các op `Exp`, `LogSoftmax`, `Clamp`, `Minimum` cùng gradient của chúng (hoà ở `Minimum` chia đôi gradient như PyTorch). Hành động không hợp lệ bị che bằng logit $-10^9$ nên xác suất bằng đúng 0. Nhiều môi trường chạy song song bằng rayon với tự động reset; actor và critic chạy suy luận qua các đồ thị đã biên dịch. `BucketedMlp` phục vụ batch kích thước thay đổi bằng cách làm tròn lên bucket; do GEMM tất định, kết quả giống hệt từng bit với đồ thị đúng kích thước.

### 4.7 Đóng gói

Checkpoint dùng đúng định dạng safetensors (header JSON căn 8 byte, dữ liệu little-endian) nên mở trực tiếp bằng thư viện Python. `Trainer` lưu cả tham số và trạng thái AdamW, nên tiếp tục huấn luyện sau khi nạp giống hệt từng bit so với không dừng. Binding Python nhận mảng float32 liên tục qua buffer protocol và trả về NumPy.

## 5. Phương pháp kiểm chứng

Mục tiêu là chứng minh được kết quả đúng chứ không chỉ thấy loss giảm. Công cụ `tools/check_stages.py` chạy các kiểm tra của từng giai đoạn lần lượt và xuất ảnh kết quả.

| Lớp kiểm tra | Nội dung | Bắt được |
|---|---|---|
| Đối chiếu nguồn độc lập | PyTorch (float64): 46 case autograd gồm cả op mới, optimizer, vòng training, PPO loss và gradient (có action mask). NumPy: GAE và động lực học CartPole. Thư viện safetensors của Python: đọc và ghi hai chiều. | Sai công thức, sai quy ước |
| Tính chất ngẫu nhiên và từng bit | Đồ thị ngẫu nhiên: đồ thị đã tối ưu và executor planned giống hệt từng bit với đồ thị gốc; GEMM so từng bit qua mọi biên và layout; bucketing giống hệt kích thước đúng; tiếp tục từ checkpoint giống hệt không dừng. | Lỗi tổ hợp, lỗi biên, rò trạng thái |
| GPU so với CPU | Từng họ op, biên tile của GEMM, 60 đồ thị ngẫu nhiên, bước training hai loss, PPO loss, AdamW trên GPU sau 5 và 100 bước. | Sai shader, sai binding |

| Lớp kiểm tra | Nội dung | Bắt được |
|---|---|---|
| Học thật | PPO giải CartPole (return tham lam ≥ 195) và hành lang có mask (≥ 0,9, tối ưu là 0,97); so với baseline PyTorch. | Lỗi chỉ lộ ra khi học |
| Đếm allocation | Global allocator đếm trong một test riêng sau khi ép worker rayon khởi động xong. | Cấp phát ẩn |
| Mutation test | 134 lần phá code ở mọi tầng, từ kernel đến shader GPU, PPO và checkpoint. | Test yếu hoặc thiếu |

| Giai đoạn | Các kiểm tra | Đạt |
|---|---|---|
| Hygiene | rustfmt, clippy | 2 |
| 1 Tensor (CPU) | unit and kernel tests | 43 |
| 2 Autograd and training | autograd, optim, nn, PyTorch parity | 36 |
| 3 IR and runtime | ir, compiler, memory, runtime, IR parity with PyTorch | 54 |
| 4 GPU backend | gpu, example: gpu_training | 15 |
| 5 Reinforcement learning | rl, RL parity, PPO solves CartPole (release) | 22 |
| 6 Library packaging | checkpoint, facade and doctests, safetensors interop, example: xor, example: checkpoint_resume, example: inspect, example: demo, build Python module, Python tests | 26 |

Mỗi dòng là một nhóm kiểm tra; các ví dụ và bước build module Python tính là một kiểm tra đạt. `clippy` (với mọi feature) và `rustfmt` không có cảnh báo.

## 5.1 Những lỗi và lỗ hổng đã tìm thấy

| Phát hiện | Nguyên nhân | Xử lý |
|---|---|---|
| Baseline PyTorch PPO học sai | Mảng quan sát dùng chung bộ nhớ với tensor và bị ghi đè trước khi sao chép: PyTorch huấn luyện trên quan sát lệch một bước. Lúc đầu nó trông kém hiệu quả mẫu gấp 8–10 lần Rust; đường cong học cho thấy return của nó giảm | Sao chép trước khi bước môi trường; hiệu quả mẫu hai bên tương đương (13–20 nghìn so với 16–35 nghìn bước) |
| Shader matmul sai thứ tự binding | wgpu báo xung đột sử dụng buffer: `out` khai báo trước `bias` và `mask` | Đặt `out` ở binding cuối như mọi kernel |
| Test GPU treo khi chạy song song | Nhiều test cùng mở thiết bị Vulkan | Dùng chung một ngữ cảnh GPU và tuần tự hoá truy cập |
| Test allocation fail ngẫu nhiên ở release | Worker rayon còn khởi động đúng lúc bắt đầu đo (không phải cấp phát trong đường chạy) | Ép worker khởi động bằng rayon::broadcast trước khi đo |
| Mutation lọt: tính hợp lệ của hành động JUMP và bootstrap giá trị của PPO | Test chỉ kiểm vị trí 0 và 4; hàm collect riêng tư nên không test nào kiểm giá trị bootstrap (PPO vẫn học được với bootstrap bằng 0) | Thêm test ở vị trí ngay trước ô mục tiêu và test trong module của ppo.rs |
| Binding Python: buffer protocol không có trong abi3 dưới Python 3.11 | Giới hạn của limited API | Bỏ abi3: module gắn với phiên bản Python lúc build |
| Số đo benchmark nhiễu | Tiến trình nền (game, trình duyệt) làm một lần đo một luồng tụt từ 120 xuống 24 GFLOP/s | Chạy xen kẽ Rust và PyTorch, lấy kết quả tốt nhất qua nhiều vòng; lưu kết quả thô |
| Script biểu đồ lỗi làm mất 15 phút đo | Regex không bắt được một dòng; dữ liệu chỉ nằm trong bộ nhớ | Lưu kết quả thô sau mỗi vòng; tuỳ chọn --reparse |
| Các lỗ hổng test ở giai đoạn trước | Vị trí epsilon của AdamW, thang khởi tạo Xavier, kiểm tra độ dài buffer, điều kiện của fusion | Thêm test; xem phần mutation |

Không tìm thấy lỗi nào trong code sản phẩm ngoài các lỗ hổng test và lỗi cài đặt baseline kể trên. Điều này chỉ đúng với những gì bộ test hiện tại bao phủ.

# 6. Kết quả

Mọi số đo là kết quả tốt nhất qua hai vòng xen kẽ Rust và PyTorch, float32. Cột kernel cũ của matmul là một lần đo trước khi viết lại. PyTorch ở dạng eager.

## 6.1 Bộ nhớ và allocation

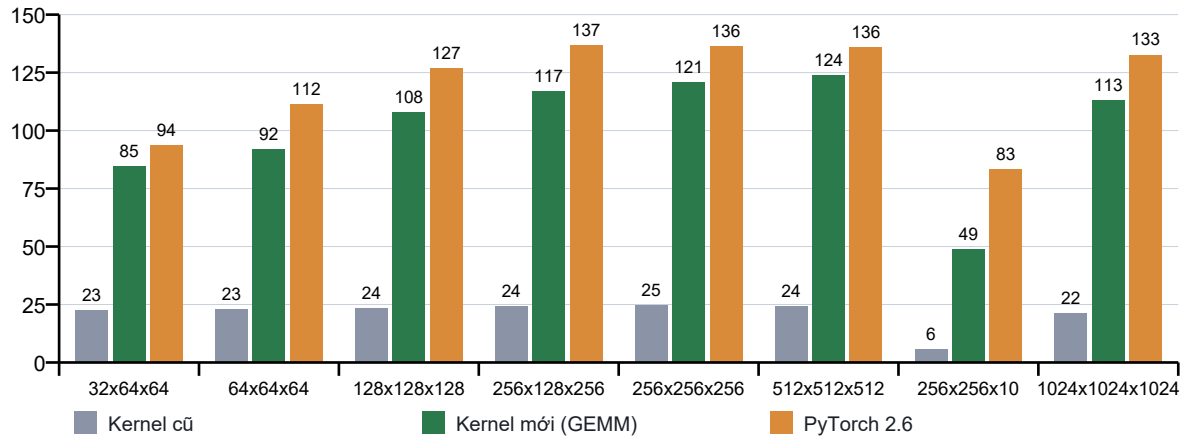
Mỗi bước training (MLP 16-64-64-4, batch 32) cấp phát 104 lần với engine động, 87 lần với executor tham chiếu và **0 lần** với executor planned (cả debug và release). Số phần tử buffer trung gian của bước training giảm qua ba lớp: đồ thị gốc, sau các pass, sau khi lập kế hoạch tái dùng buffer.

| Workload | Đồ thị gốc | Sau CSE, fusion, DCE | Sau planner | Tổng giảm |
|---|---|---|---|---|
| tiny (batch 8) | 5.606 | 2.390 | 2.144 | 2,6× |
| small (batch 32) | 35.976 | 14.344 | 10.880 | 3,3× |
| medium (batch 256) | 1.009.166 | 376.334 | 272.384 | 3,7× |

## 6.2 Matmul trên CPU (GFLOP/s)

| Shape (m,k,n) | Cũ, 1 luồng | Mới, 1 luồng | PyTorch, 1 luồng | Mới / PyTorch | Mới, đa luồng | PyTorch, đa luồng |
|---|---|---|---|---|---|---|
| 32x64x64 | 22,6 | **84,6** | 93,6 | 0,9× | 81,9 | 11,7 |
| 64x64x64 | 22,9 | **92,0** | 111,5 | 0,8× | 87,4 | 20,3 |
| 128x128x128 | 23,5 | **108,1** | 127,1 | 0,9× | 133,6 | 100,8 |
| 256x128x256 | 24,5 | **117,0** | 136,8 | 0,9× | 193,3 | 200,7 |
| 256x256x256 | 24,6 | **120,7** | 136,5 | 0,9× | 232,4 | 247,8 |
| 512x512x512 | 24,5 | **124,0** | 136,1 | 0,9× | 328,9 | 529,8 |
| 256x256x10 | 5,8 | **48,7** | 83,5 | 0,6× | 52,4 | 34,3 |
| 1024x1024x1024 | 21,5 | **113,3** | 132,8 | 0,9× | 374,0 | 496,0 |

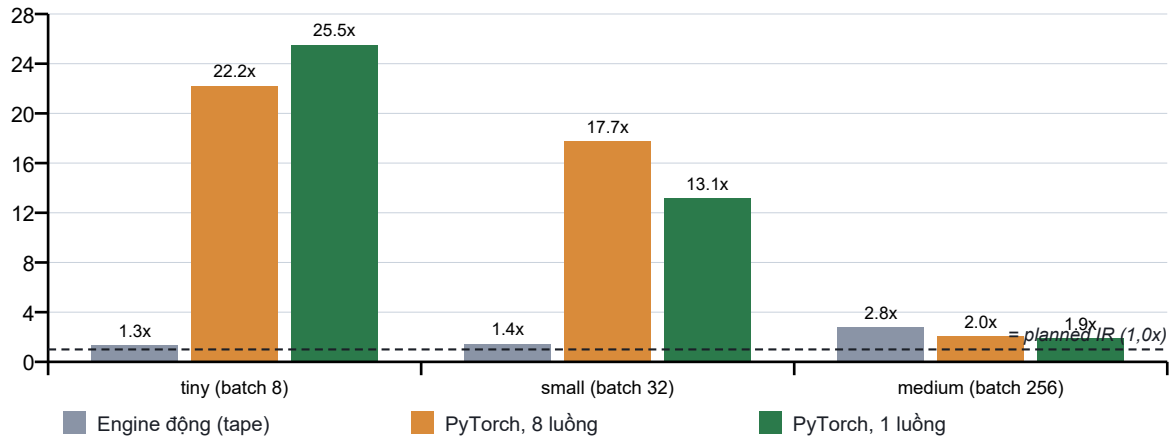
Throughput một luồng (GFLOP/s):




## 6.3 Bước training trên CPU (micro giây)

| Workload | Tape | IR tham chiếu | IR planned | PyTorch 8 luồng | PyTorch 1 luồng |
|---|---|---|---|---|---|
| tiny | 24,6 | 22,9 | **18,9** | 419,4 | 481,6 |
| small | 53,0 | 45,6 | **37,7** | 668,0 | 495,3 |
| medium | 2.920,0 | 2.697,9 | **1.058,5** | 2.149,4 | 1.980,3 |

Thời gian tương đối so với IR planned (lớn hơn 1 nghĩa là chậm hơn):

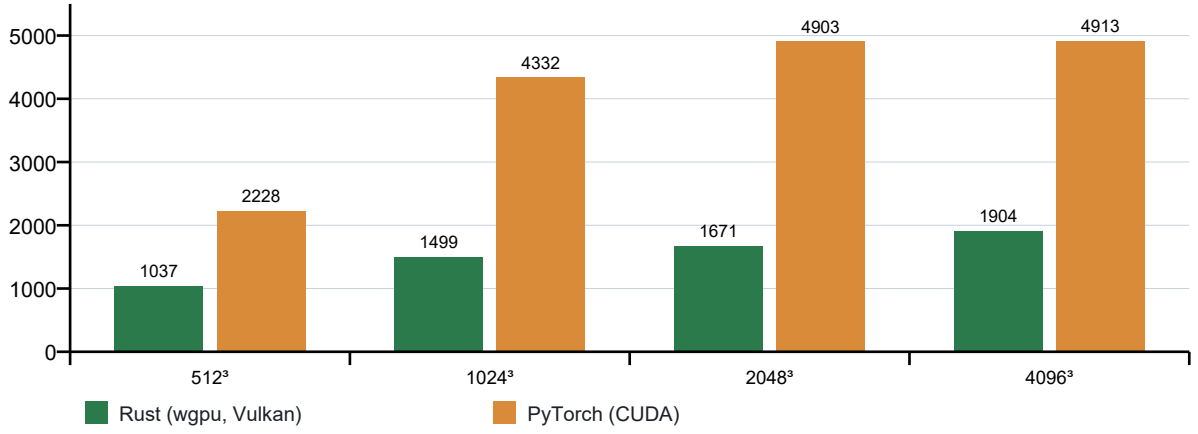



## 6.4 GPU

Cả hai bên đo với dữ liệu nằm sẵn trên thiết bị và đồng bộ khi kết thúc; bước training gồm forward, MSE, backward và AdamW.

| Matmul | Rust (wgpu), GFLOP/s | PyTorch CUDA, GFLOP/s | Rust / CUDA |
|---|---|---|---|
| 512³ | 1.037 | 2.228 | 47% |
| 1024³ | 1.499 | 4.332 | 35% |
| 2048³ | 1.671 | 4.903 | 34% |
| 4096³ | 1.904 | 4.913 | 39% |

Throughput GEMM trên GPU (GFLOP/s):




| Bước training | Rust GPU | Rust CPU | PyTorch CUDA | GPU / CPU | Rust GPU / CUDA |
|---|---|---|---|---|---|
| medium: batch 256, 128-256-256-10 | 889 µs | 1.113 µs | 1.365 µs | 1,3× | 1,5× |

| Bước training | Rust GPU | Rust CPU | PyTorch CUDA | GPU / CPU | Rust GPU / CUDA |
|---|---|---|---|---|---|
| large: batch 2048, 512-1024-1024-10 | 14.104 µs | 51.385 µs | 4.425 µs | 3,6× | 0,3× |
| xlarge: batch 4096, 1024-2048-2048-10 | 83.773 µs | 325.358 µs | 32.864 µs | 3,9× | 0,4× |

Cột cuối lớn hơn 1 nghĩa là Rust nhanh hơn PyTorch CUDA; nhỏ hơn 1 nghĩa là chậm hơn.

### 6.5 Học tăng cường: PPO trên CartPole

Cùng thuật toán và siêu tham số ở hai bên (8 môi trường, 128 bước, 4 epoch, 4 minibatch, lr 1e-3, GAE 0,95, clip 0,2), cùng tiêu chí "đã giải" (return trung bình của 3 update gần nhất ≥ 195), 3 seed. PyTorch dùng môi trường Python thuần chạy tuần tự và một luồng cho phần mạng.

| | Bước/giây | Thời gian để giải (trung vị 3 seed) | Số bước để giải |
|---|---|---|---|
| Rust (rustai) | 81.028 | 0,25 s | 16–35 nghìn |
| PyTorch, 1 luồng | 12.171 | 1,49 s | 13–20 nghìn |
| **Tỉ lệ** | **6,7×** | **6,0× nhanh hơn** | tương đương |

### 6.6 Diễn giải

- **Phần lớn lợi thế trên CPU đến từ chi phí điều phối thấp.** Bước nhỏ nhanh hơn PyTorch eager 13–25×; ở cỡ medium, nơi tính toán chiếm ưu thế, còn khoảng 1,9–2,0×. Fusion và planner đóng góp đáng kể ở cỡ medium (bước giảm từ 2,9 ms của engine động xuống 1,06 ms), nhưng chưa tách riêng phần của từng nguồn.
- **Matmul CPU gần MKL trên một luồng, kém ở đa luồng cỡ lớn.** 82–91% trên hầu hết shape; shape hẹp 256×256×10 chỉ 58%; đa luồng ở 512–1024 đạt 62–75%, nhưng ở shape nhỏ nhanh hơn PyTorch vì không tốn chi phí khởi động luồng.
- **GPU: đúng và hữu ích, chưa cạnh tranh ở cỡ lớn.** GEMM tự viết đạt 34–47% cuBLAS. Mạng nhỏ nhanh hơn PyTorch CUDA nhờ ít chi phí khởi chạy kernel; mạng lớn bị giới hạn bởi GEMM nên chậm hơn 2,5–3,2×. So với CPU của chính engine, GPU nhanh hơn khoảng 3,6–3,9× ở mạng lớn.
- **PPO: lợi thế là thông lượng, không phải hiệu quả mẫu.** Đường cong học hai bên tương đương; Rust nhanh hơn do môi trường chạy song song trong Rust và suy luận không qua Python, không do thuật toán khác.
- **Độ tin cậy của số đo.** Một máy, hai vòng xen kẽ lấy kết quả tốt nhất, tải nền thấp lúc đo. Trước khi lấy kết quả tốt nhất, các lần đo trên cùng máy lệch nhau đến 2–3× khi có tiến trình nền nặng.

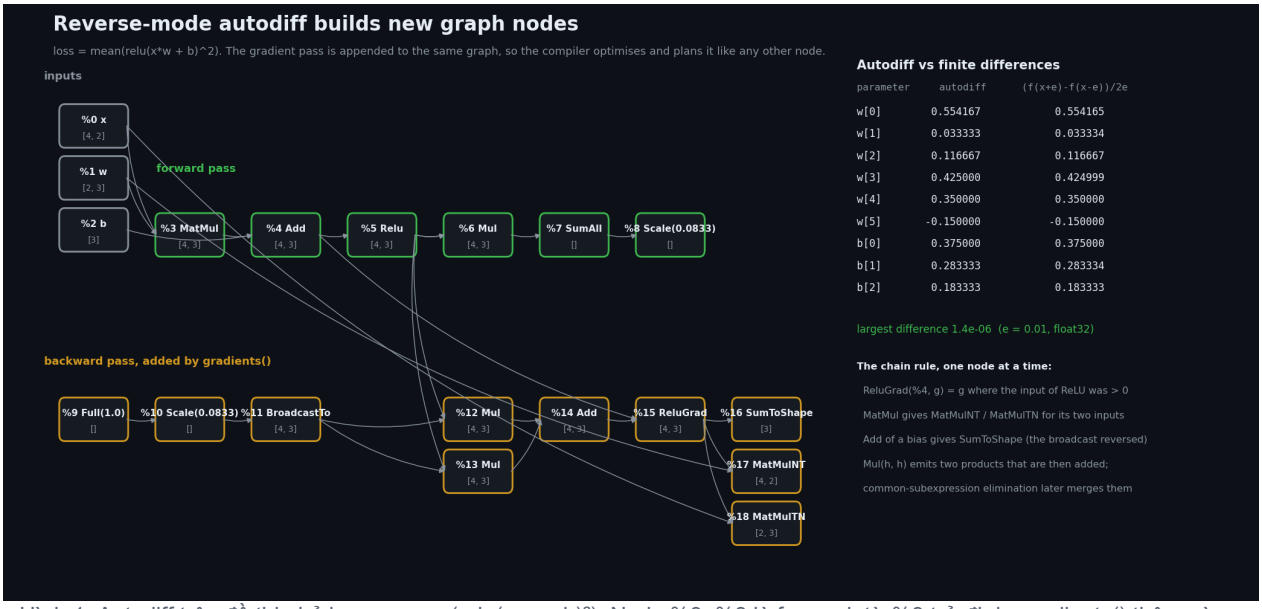
## 7. Nguyên lý hoạt động

Ba ý tưởng làm nên thư viện: autodiff sinh ra node của đồ thị, bộ lập kế hoạch quyết định mọi giá trị nằm ở đâu trước khi chạy, và GPU phát lại một danh sách kernel đã chuẩn bị sẵn. Các hình và bảng dưới đây do chính thư viện sinh ra (`python tools/explain_report.py`); bản đầy đủ nằm ở `docs/PRINCIPLES.md`.

### 7.1 Vì sao autograd hoạt động

Loss là hàm hợp $L = f_n(\dots f_2(f_1(\theta)))$, và quy tắc chuỗi nói đạo hàm của hàm hợp là tích đạo hàm của các thành phần. Có hai thứ tự nhân: chế độ xuôi cần một lượt quét cho mỗi tham số, chế độ ngược quét từ loss và mang theo một số cho mỗi node ($\partial L/\partial$node), nên **một lượt quét ngược cho gradient của mọi tham số với chi phí cỡ một lượt forward**. Mạng có hàng triệu tham số nhưng chỉ một loss, nên chế độ ngược là lựa chọn duy nhất khả thi.

Mỗi phép toán chỉ cần biết quy tắc cục bộ của nó: node y = f(a, b) nhận ȳ và chuyển lại ā = ȳ·∂y/∂a, b̄ = ȳ·∂y/∂b; node có nhiều nơi dùng thì cộng các đóng góp. Trong thư viện, đồ thị lưu theo thứ tự tô-pô, nên `gradients` duyệt ngược và **thêm node mới** tính gradient của đầu vào. Pha backward vì vậy không phải một cơ chế riêng: nó là phần tiếp theo của cùng đồ thị, trình biên dịch gộp được, planner xếp chỗ được, GPU chạy được như mọi node khác.

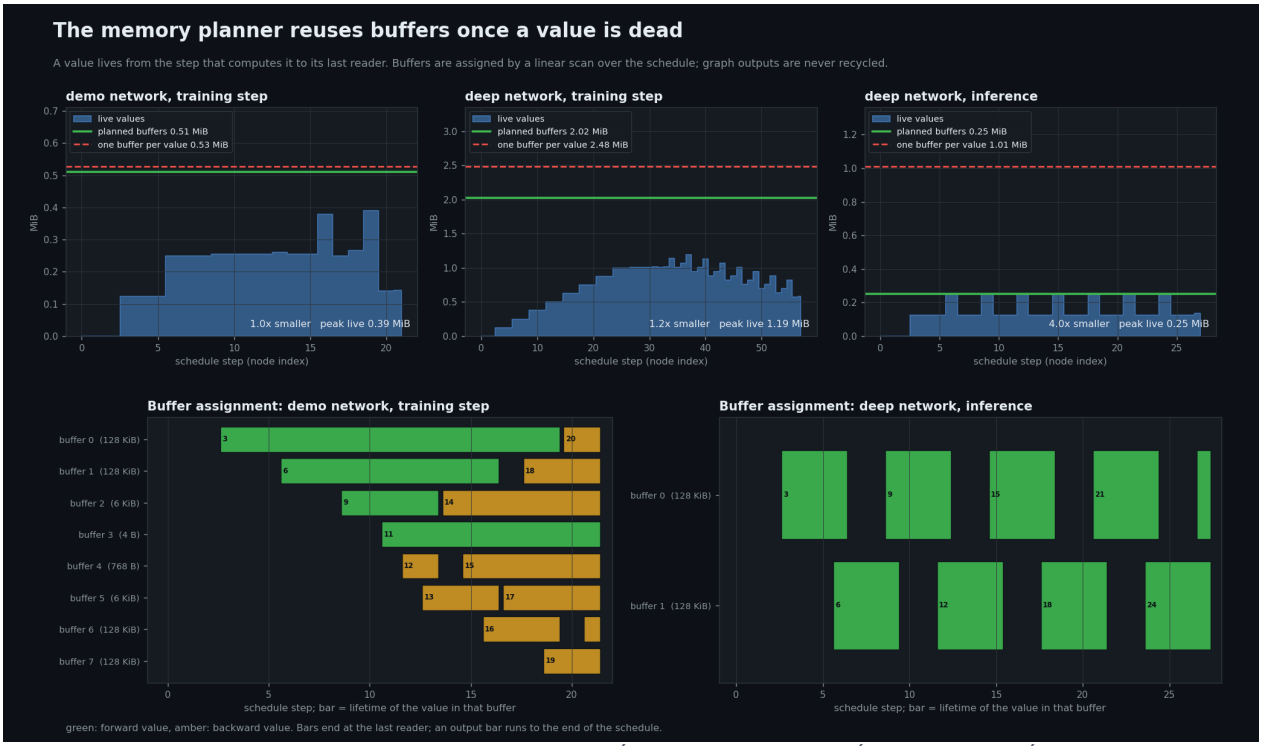



Hình 1. Autodiff trên đồ thị nhỏ loss = mean(relu(x·w + b)²). Node %3–%8 là forward; từ %9 trở đi do gradients() thêm vào. Bên phải: gradient của đồ thị so với sai phân hữu hạn.

Cách kiểm tra: ví dụ nhỏ khớp sai phân hữu hạn đến $1{,}4\cdot10^{-6}$; trên mạng demo sai lệch khoảng $10^{-3}$ theo từng tensor tham số, đó là giới hạn của chính sai phân hữu hạn (loss tính bằng float32 chia cho ε nhỏ). Kiểm tra chặt hơn là so với PyTorch float64: 46 case gradient, quỹ đạo optimizer, vòng training và PPO loss. Ở bước 0 của demo (mục 8), gradient của Rust lệch PyTorch tối đa 7e-06 so với phần tử lớn nhất.

## 7.2 Planner tiết kiệm bộ nhớ như thế nào

Sau autodiff và tối ưu, đồ thị là danh sách node với shape cố định, nên có thể quyết định mọi vị trí lưu trữ trước khi chạy. (1) Vòng đời: một giá trị tồn tại từ bước tính ra nó đến bước đọc cuối cùng; output sống đến hết. (2) Quét tuyến tính: với mỗi giá trị mới lấy buffer rảnh nhỏ nhất đủ chỗ, nếu không thì mở rộng buffer rảnh lớn nhất, nếu không thì tạo mới; khi giá trị chết, buffer được trả lại. (3) `Plan::validate` kiểm tra độc lập rằng không buffer nào chứa hai giá trị sống cùng lúc. Executor cấp phát mỗi buffer đúng một lần; bước training sau khởi động không cấp phát heap (test dùng allocator đếm).




Hình 2. Bộ nhớ trung gian (float32). Hàng trên: giá trị đang sống theo bước, so với số buffer sau lập kế hoạch và phương án mỗi giá trị một buffer. Hàng dưới: buffer nào chứa giá trị nào.

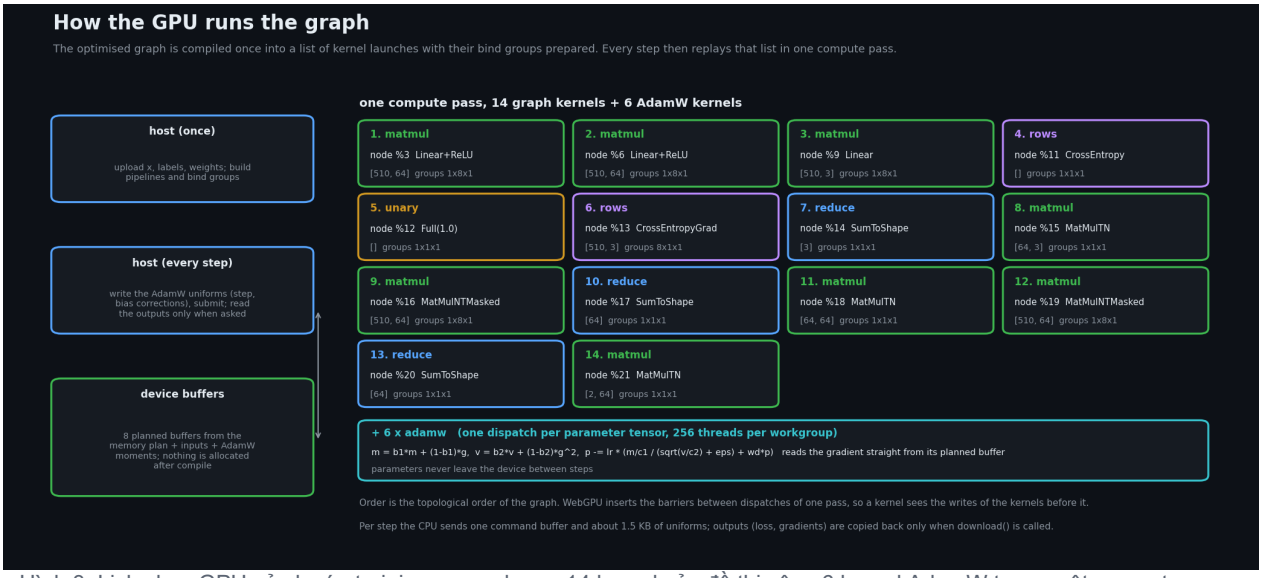
| Đồ thị | Node | Buffer | Mỗi giá trị một buffer | Sau planner | Đỉnh sống đồng thời | Giảm |
|---|---|---|---|---|---|---|
| demo network, training step | 22 | 8 | 0,53 MiB | 0,51 MiB | 0,39 MiB | 1,0× |
| deep network, training step | 58 | 20 | 2,48 MiB | 2,02 MiB | 1,19 MiB | 1,2× |
| deep network, inference | 28 | 2 | 1,01 MiB | 0,25 MiB | 0,25 MiB | 4,0× |

- **Suy luận tái dùng mạnh:** mỗi activation chết ngay khi lớp sau đọc xong, nên hai buffer đổi chỗ cho nhau suốt mạng; mức giảm tăng theo độ sâu.
- **Training tái dùng ít:** activation của forward phải sống đến khi backward cần, và gradient của mọi tham số là output nên không bao giờ được tái chế; chỉ các giá trị tạm ngắn của backward dùng chung buffer. Đây là tính chất của training ngược, không phải của planner.
- **Kế hoạch chưa tối ưu:** đỉnh sống đồng thời là chặn dưới của mọi kế hoạch; planner cao hơn 1,3× (demo) và 1,7× (mạng sâu) vì buffer là các ô kích thước cố định, không chia nhỏ được. Xếp các giá trị theo offset trong một vùng nhớ chung sẽ thu hẹp phần lớn khoảng cách này; chưa cài đặt.
- **Không phải khẳng định dùng ít bộ nhớ hơn PyTorch:** bộ cấp phát có cache của PyTorch cũng tái dùng khối đã giải phóng. Lợi ích của việc lập kế hoạch là khác: không gọi cấp phát khi training, dung lượng biết trước lúc biên dịch, và một kế hoạch dùng chung cho cả CPU lẫn GPU.

## 7.3 GPU thực thi đồ thị ra sao

Backend GPU dùng lại nguyên trình biên dịch và planner của CPU. **Biên dịch một lần:** tối ưu và lập kế hoạch; tạo một storage buffer của wgpu cho mỗi buffer đã lập kế hoạch và mỗi input; chọn kernel WGSL cho từng node (`matmul` cho MatMul, MatMulNT, MatMulTN và các node đã gộp Linear/MatMulNTMasked với bias, ReLU, mặt nạ trong epilogue; `reduce`; `rows` cho cross-entropy và log-softmax; `unary` và `binary`); tạo bind group cho mỗi node và ghi lại số workgroup. Kết quả là một danh sách (pipeline, bind group, số workgroup). **Mỗi bước:** `execute()` ghi vài uniform thay đổi (số bước của AdamW), mở một compute pass, phát lại danh sách theo thứ tự tô-pô rồi submit; WebGPU tự theo dõi hazard giữa các dispatch trong một pass. Các kernel AdamW chạy cuối: mỗi tensor tham số một dispatch, đọc gradient

trực tiếp từ buffer đã lập kế hoạch và cập nhật tham số cùng hai moment tại chỗ.

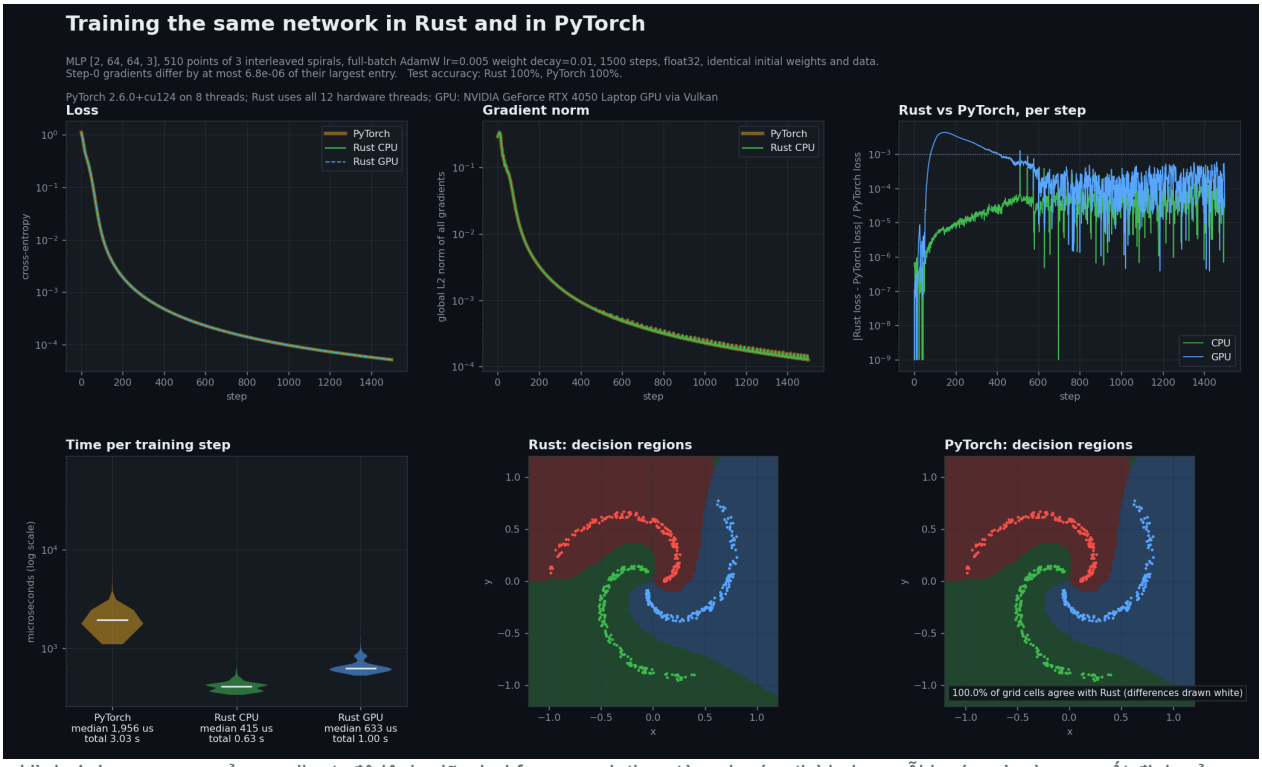



Hình 3. Lịch chạy GPU của bước training mạng demo: 14 kernel của đồ thị cộng 6 kernel AdamW trong một compute pass.

Qua bus PCIe: input và trọng số tải lên một lần; mỗi bước chỉ gửi một command buffer và khoảng 1,5 KB uniform. Đọc kết quả về là tuỳ chọn: `download()` sao chép mọi output của đồ thị (loss và toàn bộ gradient, 18 KB với mạng demo), còn tham số ở lại trên thiết bị cho đến khi gọi `read_input`. Demo ghi loss ở mỗi bước nên thời gian GPU của nó đã gồm lần tải này.

## 8. Demo: huấn luyện cạnh PyTorch

Cùng một mạng [2, 64, 64, 3], 510 điểm của ba xoắn ốc đan nhau, 1500 bước AdamW toàn batch, bắt đầu từ cùng trọng số khởi tạo (tệp safetensors) và cùng dữ liệu, huấn luyện bằng Rust trên CPU, Rust trên GPU và PyTorch. Lệnh `python tools/demo_report.py` chạy cả ba và vẽ hình.




Hình 4. Loss, norm của gradient, độ lệch giữa hai framework theo từng bước, thời gian mỗi bước và vùng quyết định của hai mạng đã huấn luyện.

| | Rust CPU | Rust GPU | PyTorch eager |
|---|---|---|---|
| Loss cuối | 5.2e-05 | xem hình | 5.2e-05 |
| Trung vị thời gian mỗi bước | 415 µs | 633 µs | 1.956 µs |
| Tổng thời gian huấn luyện | 0,63 s | 1,00 s | 3,03 s |
| Độ chính xác trên tập kiểm tra mới | 100% | 100% | 100% |

- Hai đường loss lệch nhau tối đa 0.07% tương đối (CPU); vùng quyết định của hai mạng trùng trên 100% ô lưới 100×100. Đường GPU lệch nhiều hơn ở đầu quá trình (đến khoảng $4 \cdot 10^{-3}$) vì thứ tự cộng khác.
- Thời gian mỗi bước đo quanh toàn bộ bước (forward, backward, AdamW, đọc loss). Bước GPU gồm lần tải loss và gradient mỗi bước; mạng này quá nhỏ để GPU thắng CPU của chính engine.
- Đây là số đo một máy, một lần chạy; tập kiểm tra là xoắn ốc sinh mới nên 100% chỉ nói lên bài toán dễ.

## 9. Hạn chế

- Chỉ có float32 và một tập lớp thiên về MLP: chưa có convolution, attention, LayerNorm; shape cố định lúc dựng đồ thị (`BucketedMlp` chỉ phục vụ suy luận batch thay đổi); rank tối đa là 8.
- GEMM CPU dùng AVX2/FMA, chưa dùng AVX-512 hay prefetch; GEMM GPU chỉ bằng khoảng 35–45% cuBLAS (chưa dùng tensor core hay subgroup). Kết quả FMA khác đường portable ở bit cuối và có thể khác giữa các CPU.
- Backend GPU: nhãn cross-entropy không được kiểm tra hợp lệ trên GPU (CPU có kiểm tra); reduce theo từng phần tử đầu ra nên chậm với tổng lớn (ví dụ `SumAll` trên tensor rất lớn); chỉ đã kiểm trên một card NVIDIA qua Vulkan.
- Baseline PyTorch chỉ là eager, trên cùng một máy; chưa so với `torch.compile`. Benchmark PPO chỉ có CartPole.
- Binding Python gắn với phiên bản Python lúc build và chỉ phơi bày `MlpTrainer` cùng một hàm PPO CartPole.
- Mutation test chạy thủ công, chưa nằm trong CI; nó chỉ đo độ mạnh của test, không chứng minh không còn lỗi. CI (GitHub Actions) đã viết nhưng chưa chạy trên GitHub.
- Ba giả thuyết nghiên cứu ban đầu (chuyên biệt hoá đồ thị thích ứng, runtime actor–learner chung, đồ thị fused riêng cho RL) mới có nền tảng, chưa được đánh giá riêng.

## 10. Hướng tiếp theo

| Ưu tiên | Việc | Lý do |
|---|---|---|
| 1 | GEMM GPU: tile lớn hơn, subgroup, tensor core (cooperative matrix) khi wgpu hỗ trợ | Khoảng cách lớn nhất so với PyTorch ở mạng lớn |
| 2 | Thêm LayerNorm, GELU, softmax, attention vào IR (autodiff, CPU và GPU) | Mở rộng từ MLP sang Transformer nhỏ |
| 3 | Môi trường có trạng thái lớn hơn (ví dụ trò chơi thẻ bài có action mask) và runtime actor–learner chồng lấp | Workload mục tiêu của nghiên cứu |
| 4 | Đưa check_stages, mutation test và benchmark vào CI; báo khoảng tin cậy | Hiện tái lập thủ công |
| 5 | AVX-512 và biến thể cho shape hẹp ở CPU | Phần còn lại so với MKL |

## Phụ lục A. Tái lập kết quả

```
python tools/reproduce.py [--quick]                 # một lệnh: test, demo đối chiếu PyTorch, benchmark (kết quả: t
python tools/demo_report.py                          # huấn luyện Rust và PyTorch cạnh nhau, vẽ docs/images/demo.pn
python tools/explain_report.py                       # hình của docs/PRINCIPLES.md
cargo test --workspace --exclude rt-python          # mọi test Rust (test GPU tự bỏ qua nếu không có adapter)
cargo test --release --workspace --exclude rt-python -- --ignored   # test học
python tools/check_stages.py                         # từng giai đoạn một, xuất docs/images/stages.png
python tools/benchmark_report.py [số vòng]           # đo và vẽ biểu đồ (kết quả thô: docs/benchmark-raw.json)
python tools/test_report.py [--release]              # ảnh kết quả test theo crate
python tools/build_python.py                         # build module Python
python tests/parity/generate.py                      # sinh lại fixture PyTorch/NumPy
cargo run --release -p rustai --example cartpole_ppo
cargo run --release -p rustai --features gpu --example gpu_training
```

Môi trường đo: Windows 11, Rust 1.99.1, PyTorch 2.6.0+cu124, Intel Core i5-13420H (12 luồng cho rayon, PyTorch 8 luồng mặc định), NVIDIA GeForce RTX 4050 Laptop (Vulkan cho Rust, CUDA cho PyTorch).